MODULE 12 · DOMAIN 3 - INTEGRATION

Progressive Discovery versus Monolithic Context

When to load everything up front, when to let Claude find what it needs, and how to build the hybrid most systems end up with.

DOMAIN 3 of 7 EXAM WEIGHT 19% MODULE 12 of 38

admin@sthithapragna.com

Hand over the menu, not every recipe book in the kitchen. **Unless the menu is the whole book.**

Two ways to fill the context

EVERYTHING UP FRONT

Monolithic context

- One call, no lookup step
- Nothing can be missed
- Cache-friendly when it is stable
- Pays for every token on every request
- Selection and attention suffer as it grows

LOAD ON DEMAND

Progressive discovery

- A small starting context
- Scales to thousands of tools or documents
- An extra step for each lookup
- A risk of not finding the right thing
- Only as good as the names and descriptions

THE TRADE

Monolithic trades tokens and accuracy for certainty. Discovery trades a step and a miss risk for scale.

Discovery comes in four forms

WHAT IS DISCOVERED	MECHANISM
Tools	Tool search with deferred loading
MCP servers	Code execution: tools as code files, read on demand
Procedures	Skills: metadata always, body when relevant
Knowledge	Just-in-time retrieval: keep references, load when needed

ON THE EXAM

Same idea four times: a short index in context, the detail fetched only when it is needed.

Tool search: how it works

- **Mark tools defer_loading.** They are sent, but kept out of the context window.
- **Claude searches the catalogue:** names, descriptions, argument names. Regex or BM25.
- **Matches load as needed**, typically three to five tools per request.
- **Keep the three to five most used tools loaded**, so they need no search.
- **The cache survives:** deferred tools sit outside the cached prefix.

> ON THE EXAM
>
> Search reads names and descriptions. Badly described tools are badly found.

Tool search by the numbers, October 2026

FACT	VALUE
Selection accuracy degrades beyond	30 to 50 available tools
Five common MCP servers, before any work	About 55,000 tokens of definitions
Typical reduction with tool search	Over 85%
Use it when	10+ tools, 10K+ tokens of definitions, or a growing library
Skip it when	Under 10 tools, or every tool is used on every request
Claude Code	Defers all MCP tools by default

ON THE EXAM

The thresholds will move. The rule will not: discover when the catalogue is large, growing or rarely used whole.

Code execution with MCP

- **Tools become code:** the agent reads only the definitions it needs from files.
- **Intermediate results stay in the sandbox.** Only the answer returns to context.
- **Anthropic's example:** 150,000 tokens down to 2,000, a 98.7% saving.
- **Privacy bonus:** bulk personal data need not pass through the model.
- **The cost:** a secure sandbox, resource limits and monitoring to run.

> ON THE EXAM
>
> When big data flows between tools, let code move it and let the model see the result.

When monolithic is the right answer

SIGNAL IN THE BRIEF	LEAN TOWARDS
Under about 200K tokens and stable	Load it all, and cache it
Few tools, used on nearly every request	Load them all
Each request needs most of the content	Load it all
Large, growing, or mostly unused per request	Discover on demand
Changes often	Retrieve at request time
A miss is costly	Keep it loaded, or test discovery hard

ON THE EXAM

Small, stable and always needed: put it in. Large, changing or rarely needed: discover it.

The hybrid most systems end up with

Always loaded

Core instructions
Three to five hot tools

Cached up front

Small, stable reference
Under about 200K tokens

Hybrid context

Decided per item, not per system
Measured, then adjusted

Hot items loaded, the rest found

Discovered

The long tail of tools
Skills bodies

Retrieved

Large or changing corpus
Loaded just in time

THE TAKEAWAY

Claude Code works this way: CLAUDE.md up front, files found with search tools just in time.

Discovery is not a security control

- **A deferred tool is still offered.** Claude can find it and call it.
- **Discovery fixes context and selection,** not the blast radius.
- **Remove first, then defer:** the audit from module 11 comes before tool search.
- **Measure misses:** evaluate whether the right tool or document was found.

> THE PICTURE
>
> Hiding a tool behind search is a drawer, not a lock.

Designing the context, in order

01
Remove
What no task needs.

02
Measure
Size, stability, use per request.

03
Keep hot items
Loaded and cached.

04
Defer or retrieve
The long tail and changing content.

05
Evaluate misses
Was the right thing found?

ON THE EXAM

Discovery is a design decision with a failure mode. Test the failure mode.

> After the split, Tallyhawk's integration-fault agent connects to MCP servers for logs, metrics, tickets, the code repository and three partner accounting systems: about 140 tools and 48,000 tokens of definitions.
>
> Each fault uses four to six tools, and which ones varies by case. Wrong tool choices are common.

What should Corvane recommend?

A. Enable tool search with deferred loading, keeping the few most used tools loaded

B. Keep all the tools loaded, and cache their definitions

C. Split it into seven agents, one per MCP server, with a coordinator

D. Remove partner-system tools that were not used last month

◆ PAUSE AND DECIDE

What should Corvane recommend?

A Enable tool search with deferred loading, keeping the few most used tools loaded

A large catalogue, a small and varying slice per case: discovery's home ground.

B Keep all the tools loaded, and cache their definitions

Caching cuts cost, but 140 tools still degrade selection.

C Split it into seven agents, one per MCP server, with a coordinator

One fault spans several servers. Splitting breaks shared context.

D Remove partner-system tools that were not used last month

Usage is the wrong test. Next month's fault may need them.

THE SIGNAL

Many tools, few used per case, and the set varies: defer them and let Claude search.

> Mereside's complaints triage agent has six tools, about 2,500 tokens of definitions, and uses nearly all of them on every complaint.
>
> A developer proposes putting all six behind tool search, because Anthropic recommends it for agents with tools.

What should Corvane recommend?

A Defer all six tools behind tool search, as proposed

B Keep all six loaded, and cache them with the system prompt

C Defer five tools, and keep only the most used one loaded

D Move the tools to code execution over an MCP server

◆ PAUSE AND DECIDE

What should Corvane recommend?

A Defer all six tools behind tool search, as proposed

Adds a search step to every complaint, and saves almost nothing.

B Keep all six loaded, and cache them with the system prompt

Few, small and always used: monolithic, made cheap by caching.

C Defer five tools, and keep only the most used one loaded

Still a search for tools needed every time.

D Move the tools to code execution over an MCP server

A sandbox to run, for no data-volume problem.

THE SIGNAL

Under ten tools, all used every time: load them. Discovery is for catalogues, not toolkits.

> Fernhollow Energy, a UK supplier, runs an agent that reconciles meter readings. Each run pulls about 40,000 readings through one MCP tool, finds anomalies, and writes them to a case system through another.
>
> Every reading passes through the model's context, so each run is slow and costly. The readings include customer personal data.

What should Corvane recommend?

A Move to the model tier with the largest context window

B Summarise the readings before passing them to the model

C Let the agent write sandboxed code that calls the tools, filters there, and returns only the anomalies

D Split the run into one agent per meter region

◆ PAUSE AND DECIDE

What should Corvane recommend?

A. Move to the model tier with the largest context window

Every current tier has the same window. Cost and latency stay.

B. Summarise the readings before passing them to the model

Anomalies are exactly what a summary loses.

C. Let the agent write sandboxed code that calls the tools, filters there, and returns only the anomalies

Bulk data stays in the sandbox; the model sees the result.

D. Split the run into one agent per meter region

Every reading still passes through a model, many times over.

THE SIGNAL

Large intermediate results between tools: move them with code, not through context.

> Tallyhawk wants its engineering assistant to work across the monorepo and about 30 services, several million tokens of code that changes daily.
>
> Each team also has written conventions and architecture notes, a few thousand tokens each, that every task must respect.

How should Corvane design the assistant's context?

A Load the conventions and notes up front, and let the agent search and read code on demand

B Load as much of the monorepo as fits, and cache it for the day

C Index all the code in a vector store, and inject the top matches before every task

D Load nothing up front, and let the agent discover the conventions as it works

◆ PAUSE AND DECIDE

How should Corvane design the assistant's context?

A Load the conventions and notes up front, and let the agent search and read code on demand

Small, always-needed rules loaded; huge, changing code found just in time.

B Load as much of the monorepo as fits, and cache it for the day

Mostly irrelevant tokens, and stale by lunchtime.

C Index all the code in a vector store, and inject the top matches before every task

Guesses relevance before the task is understood, from an index that drifts.

D Load nothing up front, and let the agent discover the conventions as it works

Rules every task must follow should never depend on being found.

THE SIGNAL

Always-needed and small: load it. Huge and changing: let the agent find it. That is the hybrid.

> Tallyhawk's billing agent still has account-deletion and bulk-refund tools that none of its tasks need.
>
> An engineer argues they can stay: with tool search enabled they will be deferred, so they no longer clutter the context.

What should Corvane tell the engineer?

A Agreed, because deferred tools are not visible to the model

B Deferred tools can still be found and called, so tools no task needs must be removed

C Deferring them would break the prompt cache for the whole agent

D Deferred tools still cost their full input tokens on every request

◆ PAUSE AND DECIDE

What should Corvane tell the engineer?

A Agreed, because deferred tools are not visible to the model

They are one search away. Hidden from context is not removed.

B Deferred tools can still be found and called, so tools no task needs must be removed

Discovery fixes clutter, not blast radius. Least privilege still applies.

C Deferring them would break the prompt cache for the whole agent

False. Deferred tools sit outside the cached prefix.

D Deferred tools still cost their full input tokens on every request

False. Only tools that search loads are counted in context.

THE SIGNAL

A tool no task needs is removed, not hidden. Discovery is a drawer, not a lock.

> Quillon's prior-authorisation agent uses tool search over about 200 tools from the health record, insurer portals and document services.
>
> The insurer-criteria tools, named like ic_lookup_v2 with one-line descriptions, are needed on nearly every case. Evaluation shows the agent often fails to find them.

Which two changes most directly reduce these misses? Choose two.

A Turn off tool search and load all the tools on every request

B Give the criteria tools clear, namespaced names and full descriptions

C Keep the criteria tools loaded rather than deferred

D Raise the effort level to the maximum

E Return more matches from every search

◆ PAUSE AND DECIDE

Which two changes most directly reduce these misses? Choose two.

A. Turn off tool search and load all the tools on every request

Two hundred tools: the selection problem returns, at higher cost.

B. Give the criteria tools clear, namespaced names and full descriptions

Search reads names and descriptions. Make them findable.

C. Keep the criteria tools loaded rather than deferred

Needed on nearly every case: hot tools stay loaded.

D. Raise the effort level to the maximum

More thinking cannot find a tool its description hides.

E. Return more matches from every search

More noise in context, and a vague tool still ranks badly.

THE SIGNAL

Discovery misses: fix what search reads, and stop searching for what is always needed.

In five lines

- **Monolithic** buys certainty with tokens; **discovery** buys scale with a step and a miss risk.
- **Four forms:** tool search, code execution with MCP, Skills, just-in-time retrieval.
- **Load it all** when it is small, stable and used on nearly every request.
- **Most systems are hybrid:** hot items loaded, the long tail found.
- **Discovery is a drawer, not a lock.** Remove first, then defer, then test for misses.

COMING UP

Next: choosing the integration mechanism. MCP, a direct API, or agent-to-agent.